

CASE STUDY DOCUMENT

Task Management and Collaboration Tool

FlowBoard

Organise Work. Collaborate Seamlessly. Deliver Faster.

Version 1.0 | 2026

1. Synopsis

FlowBoard is a full-stack Task Management and Collaboration Tool inspired by Trello. It enables teams to organise work visually using a Kanban-style board system with draggable cards, manage projects across multiple workspaces, assign tasks to team members, track progress with checklists and due dates, and collaborate through comments, file attachments, and labels — all in a single, unified platform.

The platform supports three primary roles: Members (team contributors), Board Owners / Workspace Admins (project leads), and Platform Administrators. Members can join workspaces, create and manage boards and lists, create cards with rich metadata (priority, due date, assignee, labels, checklists), and collaborate via threaded comments and file attachments. Board Owners manage access control, reorder lists and cards via drag-and-drop, and monitor project progress. Platform Administrators oversee the entire system — managing users, workspaces, boards, analytics, and audit logs.

FlowBoard is architected as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, workspace management, board management, list/column management, card/task management, comment and attachment handling, label and checklist management, and notifications — all integrated through a Spring MVC website controller layer.

2. Detailed Requirements

2.1 General Requirements

- Users can view public boards without logging in.
- Users must register or log in (email/password or OAuth) to create boards, cards, or collaborate.
- Role-based access: Member, Board Owner/Workspace Admin, and Platform Administrator.
- All users can update their profile, username, avatar, and password.
- In-app and email notifications are dispatched for assignments, mentions, due-date reminders, and card movements.
- Platform Admin panel provides full management over users, workspaces, boards, and analytics.

2.2 Member Requirements

- Register and log in with email/password or via Google/GitHub OAuth.
- Create and manage personal or team workspaces; invite other users to workspaces.
- Create boards within a workspace with a name, background, and visibility setting (Public or Private).
- Add and remove lists (columns) on a board; rename, reorder, or archive lists.
- Create cards within a list — specifying title, description, priority (Low/Medium/High/Critical), due date, and start date.

- Move cards between lists (drag-and-drop); reorder cards within the same list.
- Assign a card to any board member; reassign or unassign as needed.
- Attach colour-coded labels to cards for quick visual categorisation.
- Create checklists on a card with individual items; mark items as complete to track sub-task progress.
- Upload file attachments (images, PDFs, documents) to cards.
- Add threaded comments on cards; reply to existing comments; edit or delete own comments.
- Archive cards and lists (soft-delete, recoverable); permanently delete archived items.
- Search for cards across boards by title, label, or assignee.
- Receive notifications for: card assignment, @mention in comment, due date approaching, card moved.

2.3 Board Owner / Workspace Admin Requirements

- Create, edit, and delete workspaces; set workspace visibility (Public or Private).
- Add members to a workspace with a role (Admin or Member); update or remove members.
- Create, edit, close, and delete boards within a workspace.
- Manage board membership: add/remove board members and set board-level roles (Observer, Member, Admin).
- Access board-level analytics: card counts per list, overdue cards, completion rate, and member activity.
- Reorder lists via drag-and-drop; move lists between boards in the same workspace.
- View and restore archived cards and lists.

2.4 Platform Administrator Requirements

- View and manage all user accounts: suspend, reactivate, or permanently delete users.
- View and manage all workspaces and boards across the platform.
- Access platform-wide analytics: total users, boards, cards created, and active teams.
- View audit logs showing all significant actions (card creation, member role changes, deletions).
- Generate and export activity reports for workspaces or individual users.
- Send platform-wide broadcast notifications to all or targeted user groups.
- View all overdue cards across the platform for SLA monitoring.

2.5 Card / Task Requirements

- Cards are ordered by position within a list; position changes are persisted on every drag-and-drop.
- Each card supports: title, rich-text description, priority (Low/Medium/High/Critical), status (To Do/In Progress/In Review/Done), due date, start date, cover colour, and assignee.
- A card can have multiple labels (from the board's label set), multiple checklist items, and multiple file attachments.

- Card activity log records every change (assignment, status, due date, move) with actor and timestamp.
- Cards are flagged as overdue when the current date exceeds the due date and the status is not Done.

2.6 Notification Requirements

- Notifications include: card assigned to you, @mention in comment, due date approaching (1 day, 1 hour), card moved to Done, comment reply.
- Each notification carries the actorId, recipientId, related entity (cardId/boardId), and a deep-link URL.
- Unread notification badge count is displayed in real time in the top navigation bar.
- Bulk notification dispatch is available for admin broadcast and system-generated reminders.
- Email notifications are sent for critical events (direct assignment, overdue card).

3. Use Case Diagram

The diagram below captures all actors and use cases within the FlowBoard platform — spanning authentication, workspace and board management, list and card operations, collaboration (comments, attachments, labels, checklists), notifications, and administrative controls.

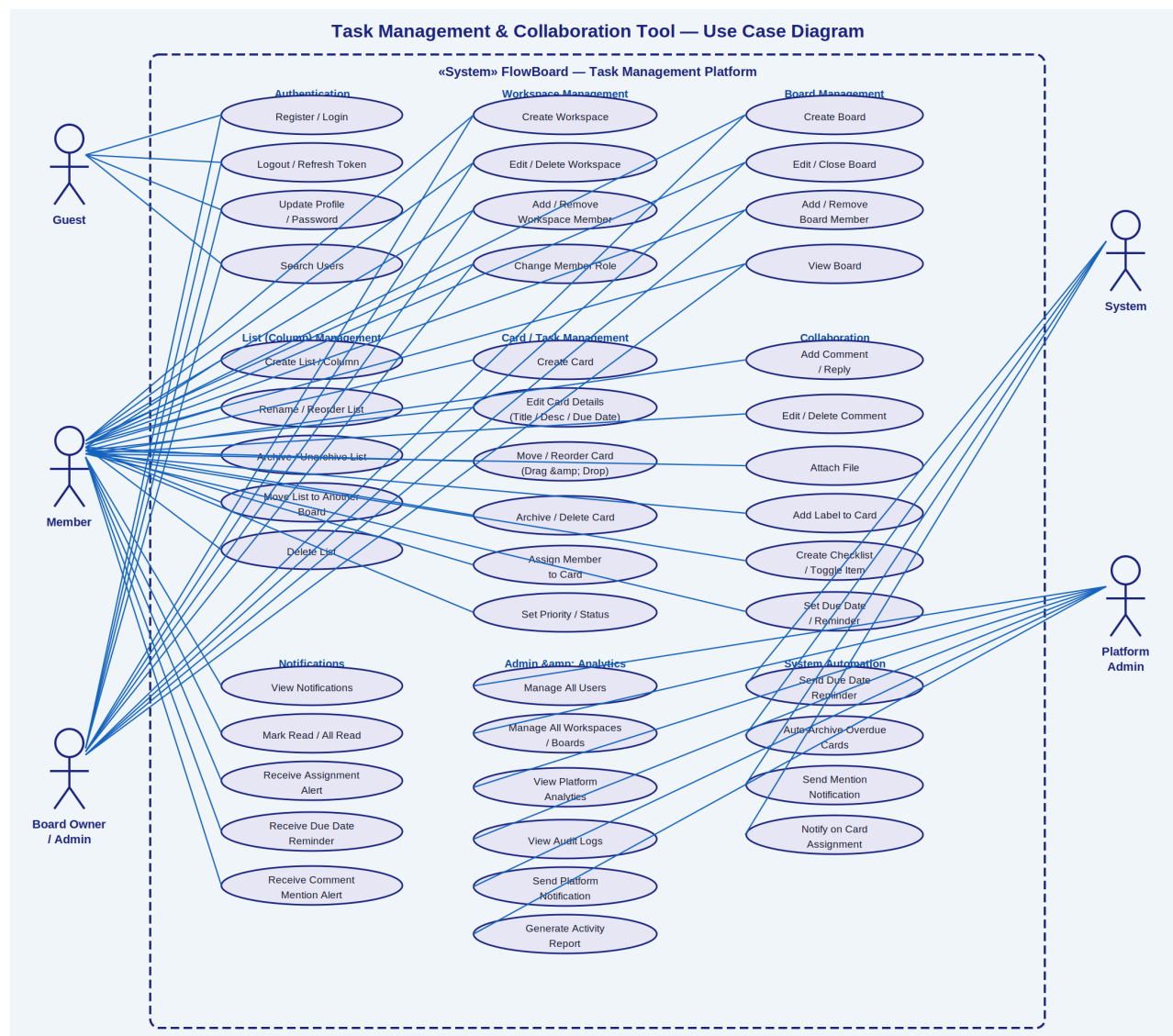


Figure 1: FlowBoard Task Management Platform — Complete Use Case Diagram

3.1 Actors

Actor	Description
Guest	Unauthenticated visitor who can view public boards
Member	Registered team contributor who creates boards, cards, and collaborates
Board Owner / Admin	Workspace or board administrator who manages access, settings, and analytics
Platform Admin	System administrator with full management over all users, workspaces, and boards
System	Automated component handling due-date reminders, overdue detection, and notification dispatch

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Login	Guest, Member	Create an account or authenticate via email or OAuth
Update Profile	Member	Change username, avatar, bio, or password
Search Users	Member	Find users by name or username to invite to workspace/board
Create Workspace	Member	Create a named workspace with optional description and visibility
Edit / Delete Workspace	Board Owner	Modify workspace settings or permanently remove it
Add / Remove Member	Board Owner	Invite a user to or remove them from a workspace or board
Change Member Role	Board Owner	Update a workspace or board member's permission role
Create Board	Member, Board Owner	Create a Kanban board within a workspace with background and visibility
Edit / Close Board	Board Owner	Modify board settings or close it to prevent further changes
View Board	Guest, Member	Open a board and view all its lists and cards
Create List / Column	Member	Add a new vertical column to a board
Rename / Reorder List	Member	Change list name or drag it to a new horizontal position
Archive / Unarchive List	Member	Soft-remove a list from view; restore it from the archive

Use Case	Actor(s)	Description
Move List	Board Owner	Move a list to a different board within the same workspace
Create Card	Member	Add a task card to a list with title and optional metadata
Edit Card Details	Member	Update card title, description, due date, priority, or status
Move / Reorder Card	Member	Drag a card to a different list or reorder within the same list
Archive / Delete Card	Member	Soft-archive a card or permanently delete an archived card
Assign Member to Card	Member	Set a board member as the assignee responsible for a card
Set Priority / Status	Member	Tag a card with a priority level and update its workflow status
Add Comment / Reply	Member	Post a top-level comment or threaded reply on a card
Edit / Delete Comment	Member	Modify or remove own comments on a card
Attach File	Member	Upload an image, PDF, or document to a card
Add Label to Card	Member	Tag a card with one or more colour-coded labels from the board set
Create Checklist	Member	Add a sub-task checklist with items to a card
Toggle Checklist Item	Member	Mark a checklist item as complete or incomplete
View Notifications	Member	Read in-app alerts for assignments, mentions, and reminders
Mark Read / All Read	Member	Mark individual or all notifications as read
Receive Assignment Alert	Member, System	Get notified when assigned to a card
Receive Due Date Reminder	Member, System	Automated alert 1 day and 1 hour before card due date
Manage All Users	Platform Admin	View, suspend, reactivate, or delete any user account
Manage Workspaces / Boards	Platform Admin	View, audit, or remove any workspace or board on the platform
View Platform Analytics	Platform Admin	Access total users, boards, cards, and team activity metrics
View Audit Logs	Platform Admin	Inspect timestamped log of all significant platform actions
Generate Activity Report	Platform Admin	Export usage and activity data for a workspace or user

Use Case	Actor(s)	Description
Send Platform Notification	Platform Admin	Broadcast an alert to all or targeted user groups
Auto-Archive Overdue Cards	System	Automatically flag or archive cards past due date with no activity
Send Mention Notification	System	Dispatch alert to @mentioned users in comments
Notify on Card Assignment	System	Send notification to user when they are assigned a card

4. Class Diagrams — All Services

FlowBoard follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). The ten class diagrams below detail every service in the platform.

4.1 Auth / User-Service

The Auth/User-Service is the security gateway for the entire platform. It handles user registration, login, JWT token generation and validation, OAuth2 integration (Google/GitHub), profile management, password changes, account deactivation, and user search. Every protected resource is gated through this service's token validation logic, and every other service resolves user identities through it.

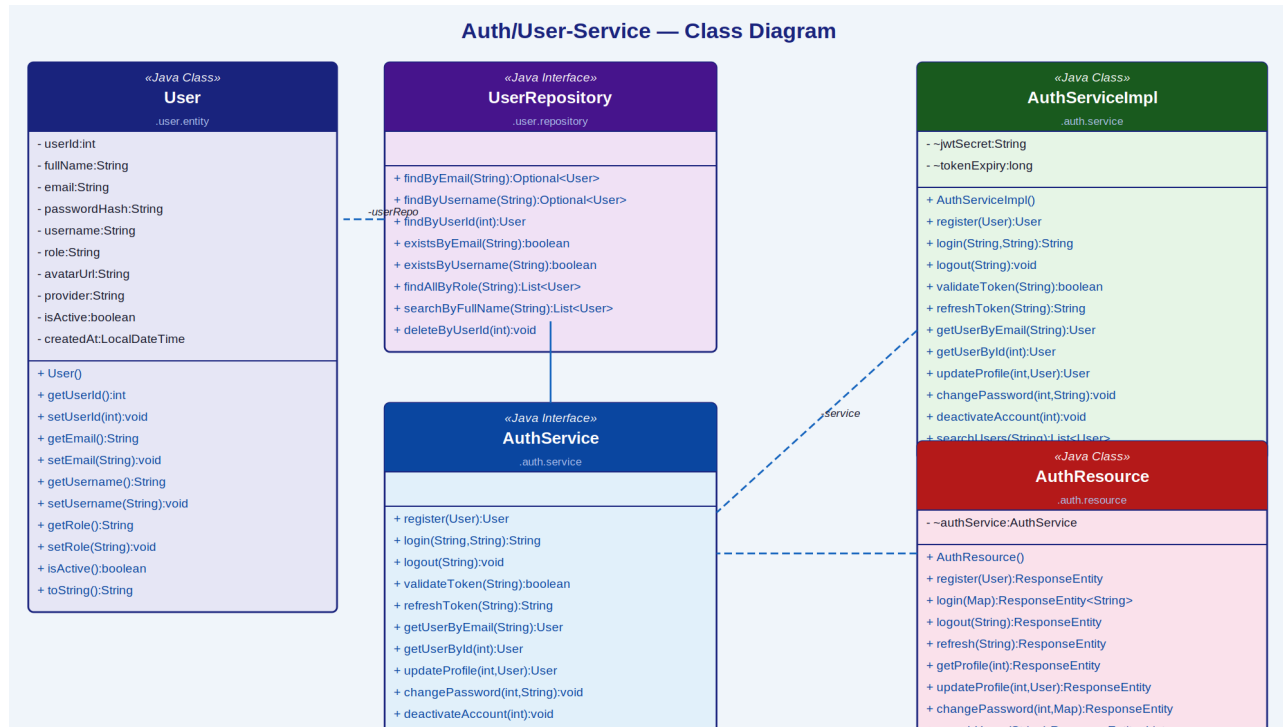


Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores userId, fullName, email, passwordHash, username, role, avatarUrl, provider (OAuth), isActive, createdAt
UserRepository	Java Interface	findByEmail(), findByUsername(), findById(), existsByEmail(), existsByUsername(), findAllByRole(), searchByFullName(), deleteById()
AuthServiceImpl	Java Class	Implements register, login, logout, JWT generation and validation, OAuth2 handling, profile update, password change, account deactivation, user search
AuthService	Java Interface	Declares register(), login(), logout(), validateToken(), refreshToken(), getUserByEmail(), getUserById(), updateProfile(), changePassword(), deactivateAccount(), searchUsers()

Component	Type	Responsibility
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/search, /auth/deactivate

4.2 Workspace-Service

The Workspace-Service manages the top-level organisational unit of the platform. A Workspace is owned by one user and can contain multiple boards. WorkspaceMembers associate users with a workspace and carry a role (Admin or Member). The service enforces membership rules, exposes workspaces accessible to a given user, and manages all workspace-level CRUD and member operations.

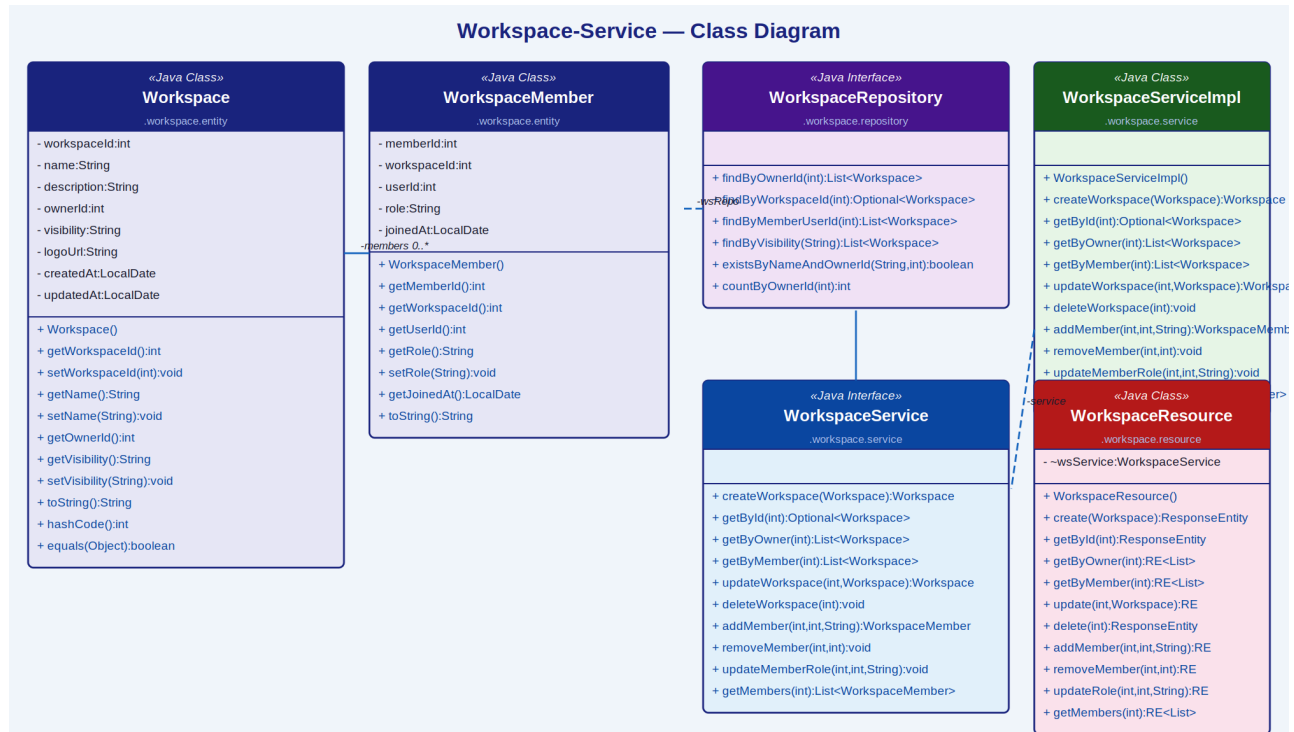


Figure 3: Workspace-Service — Class Diagram

Component	Type	Key Attributes / Methods
Workspace	Java Class (Entity)	workspaceId, name, description, ownerId, visibility (PUBLIC/PRIVATE), logoUrl, createdAt, updatedAt
WorkspaceMember	Java Class (Entity)	memberId, workspaceId, userId, role (ADMIN/MEMBER), joinedAt — linked to Workspace (0..*)
WorkspaceRepository	Java Interface	findByOwnerId(), findByWorkspaceId(), findByMemberUserId(), findByVisibility(), existsByNameAndOwnerId(), countByOwnerId()
WorkspaceServiceImpl	Java Class	createWorkspace(), getById(), getByOwner(), getByMember(), updateWorkspace(), deleteWorkspace(), addMember(), removeMember(), updateMemberRole(), getMembers()

Component	Type	Key Attributes / Methods
WorkspaceService	Java Interface	Declares all workspace CRUD and member management operations
WorkspaceResource	Java Class (REST)	Exposes /workspaces endpoints: POST (create), GET (by id/owner/member), PUT (update), DELETE, POST/DELETE (members), PUT (member role)

4.3 Board-Service

The Board-Service manages Kanban boards within workspaces. Each board has a background, visibility setting, and a closed flag. BoardMembers carry board-level roles (Observer, Member, Admin) enabling fine-grained access control within a board independent of workspace membership. The service supports board creation, editing, closing, deletion, and member management.

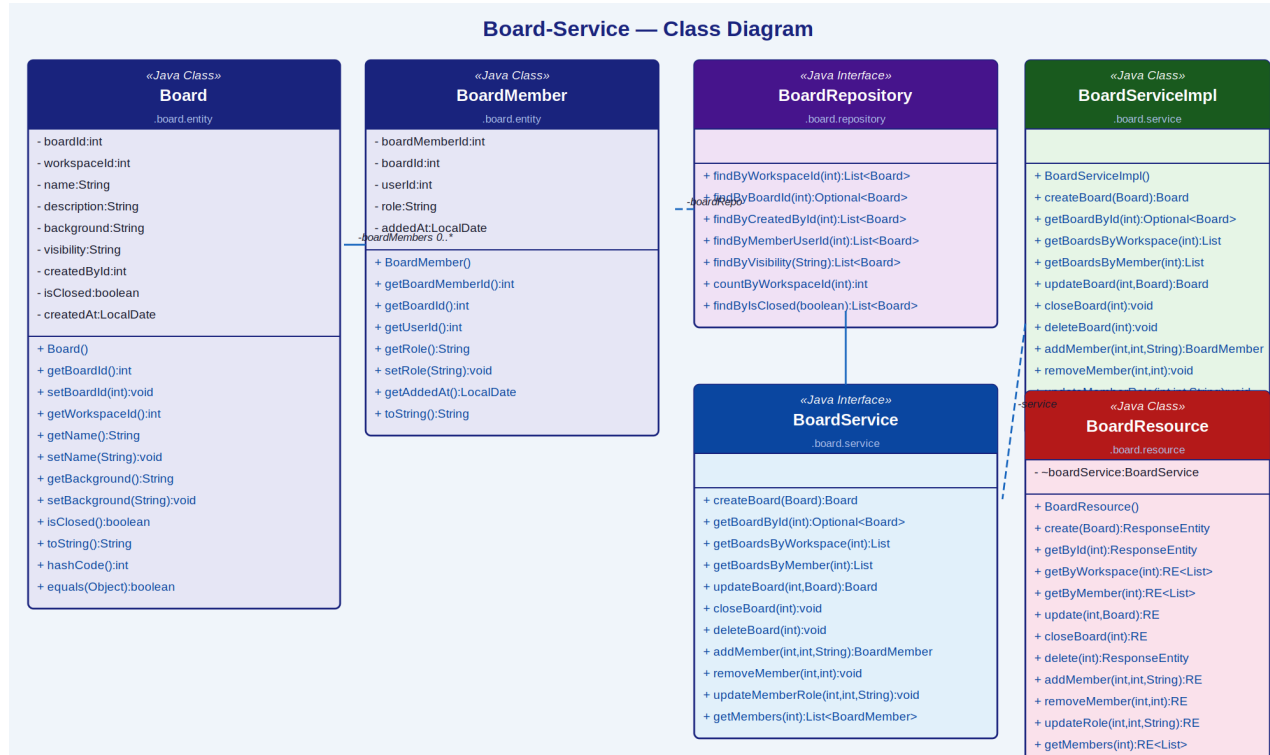


Figure 4: Board-Service — Class Diagram

Component	Type	Key Attributes / Methods
Board	Java Class (Entity)	boardId, workspaceId, name, description, background, visibility, createdById, isClosed, createdAt
BoardMember	Java Class (Entity)	boardMemberId, boardId, userId, role (OBSERVER/MEMBER/ADMIN), addedAt — linked to Board (0..*)
BoardRepository	Java Interface	findByWorkspaceId(), findByBoardId(), findByCreatedById(), findByMemberUserId(), findByVisibility(), countByWorkspaceId(), findByIsClosed()
BoardServiceImpl	Java Class	createBoard(), getBoardById(), getBoardsByWorkspace(), getBoardsByMember(), updateBoard(), closeBoard(), deleteBoard(),

Component	Type	Key Attributes / Methods
		addMember(), removeMember(), updateMemberRole(), getMembers()
BoardService	Java Interface	Declares all board CRUD, state management, and member operations
BoardResource	Java Class (REST)	Exposes /boards endpoints: POST (create), GET (by id/workspace/member), PUT (update/close), DELETE, POST/DELETE/PUT (members)

4.4 List / Column-Service

The List/Column-Service manages the vertical columns (lists) on a Kanban board. Each TaskList has a position field that determines its left-to-right display order. The service supports creating, renaming, reordering (drag-and-drop position updates for all sibling lists), archiving, unarchiving, and moving lists between boards. Position changes are committed atomically via a batch update.

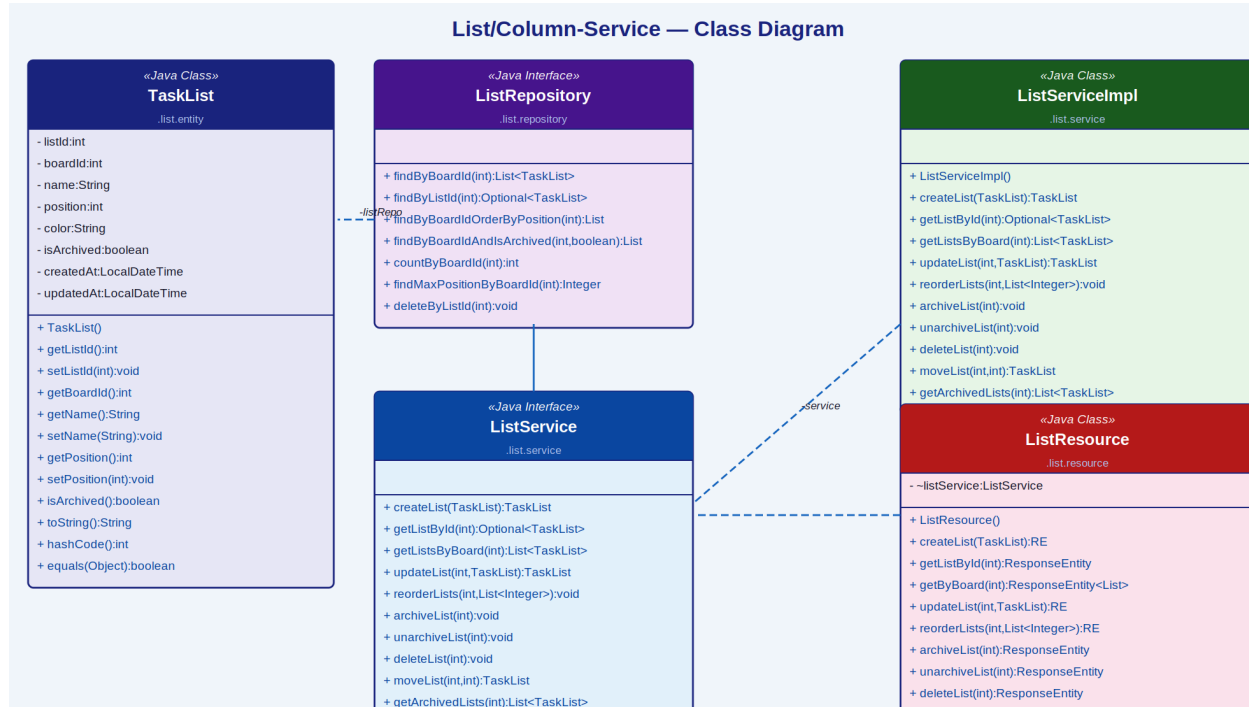


Figure 5: List/Column-Service — Class Diagram

Component	Type	Key Attributes / Methods
TaskList	Java Class (Entity)	listId, boardId, name, position, color, isArchived, createdAt, updatedAt
ListRepository	Java Interface	findByBoardId(), findByListId(), findByBoardIdOrderByPosition(), findByBoardIdAndIsArchived(), countByBoardId(), findMaxPositionByBoardId(), deleteByListId()
ListServiceImpl	Java Class	createList(), getListById(), getListsByBoard(), updateList(), reorderLists(), archiveList(), unarchiveList(), deleteList(), moveList(), getArchivedLists()
ListService	Java Interface	Declares all list CRUD, position management, archival, and board-transfer operations
ListResource	Java Class (REST)	Exposes /lists endpoints: POST (create), GET (by board/id/archived), PUT (update/reorder/move), POST (archive/unarchive), DELETE

4.5 Card / Task-Service

The Card/Task-Service is the core of the platform. It manages task cards with rich metadata: title, description, priority, status, due date, start date, assignee, cover colour, and position. Cards are ordered by position within their list; moving a card updates positions of adjacent cards. The service flags overdue cards and supports multi-criteria filtering by assignee, due date, priority, and status. Every card change is logged in the card activity feed.

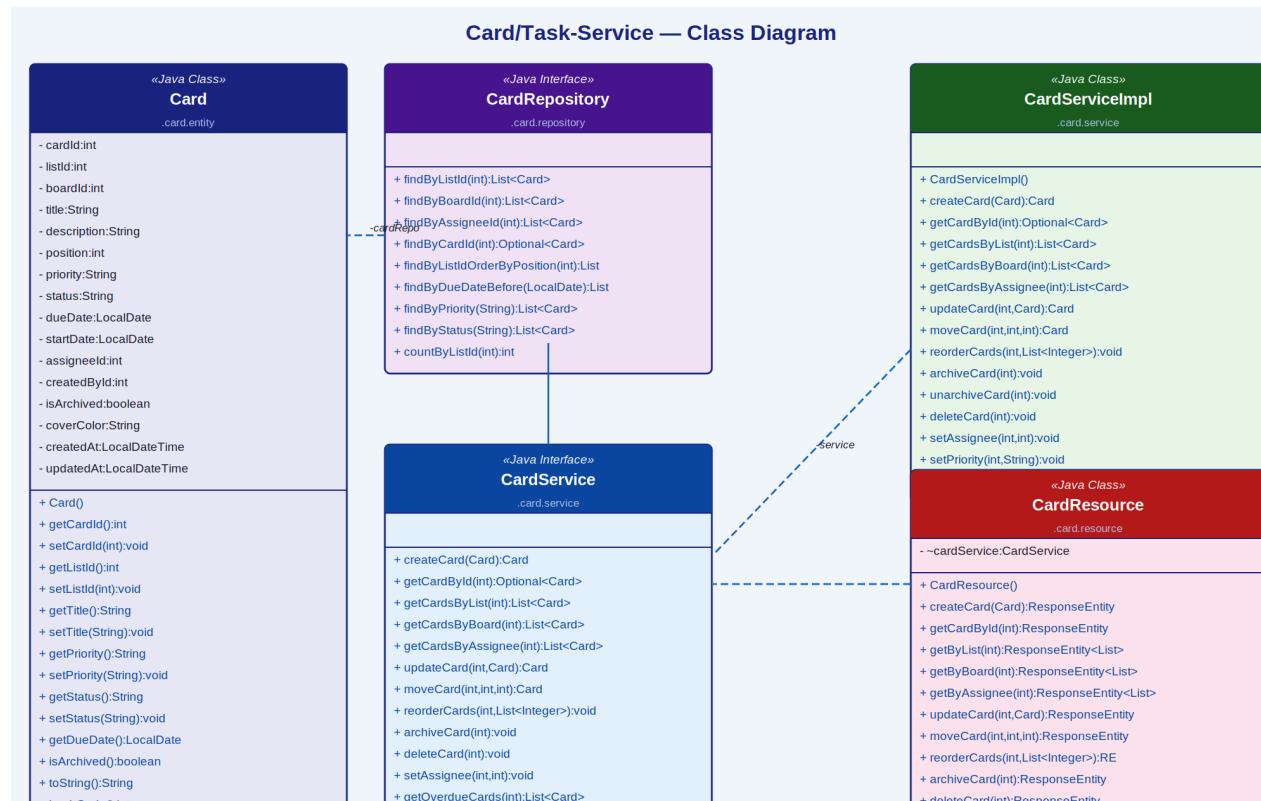


Figure 6: Card/Task-Service — Class Diagram

Component	Type	Key Attributes / Methods
Card	Java Class (Entity)	cardId, listId, boardId, title, description, position, priority (LOW/MEDIUM/HIGH/CRITICAL), status (TO_DO/IN_PROGRESS/IN_REVIEW/DONE), dueDate, startDate, assigneeId, createdBy, isArchived, coverColor, createdAt, updatedAt
CardRepository	Java Interface	findByListId(), findByBoardId(), findByAssigneeId(), findByCardId(), findByListIdOrderByPosition(), findByDueDateBefore(), findByPriority(), findByStatus(), countByListId()
CardServiceImpl	Java Class	createCard(), getCardById(), getCardsByList(), getCardsByBoard(), getCardsByAssignee(), updateCard(), moveCard(), reorderCards(),

Component	Type	Key Attributes / Methods
		archiveCard(), unarchiveCard(), deleteCard(), setAssignee(), setPriority(), getOverdueCards()
CardService	Java Interface	Declares all card CRUD, move/reorder, archival, assignment, priority, and overdue detection operations
CardResource	Java Class (REST)	Exposes /cards endpoints: POST (create), GET (by list/board/assignee/id/overdue), PUT (update/move/reorder/assignee/priority), POST (archive/unarchive), DELETE

4.6 Comment & Attachment-Service

The Comment & Attachment-Service powers collaboration on cards. Comments support two-level threading via the parentCommentId field. The service enforces soft-delete so moderation history is preserved. Attachments link uploaded files (stored in S3 or a CDN) to cards, recording file type, size, and upload timestamp. Every new comment or attachment triggers a Notification-Service call to alert card watchers and @mentioned users.

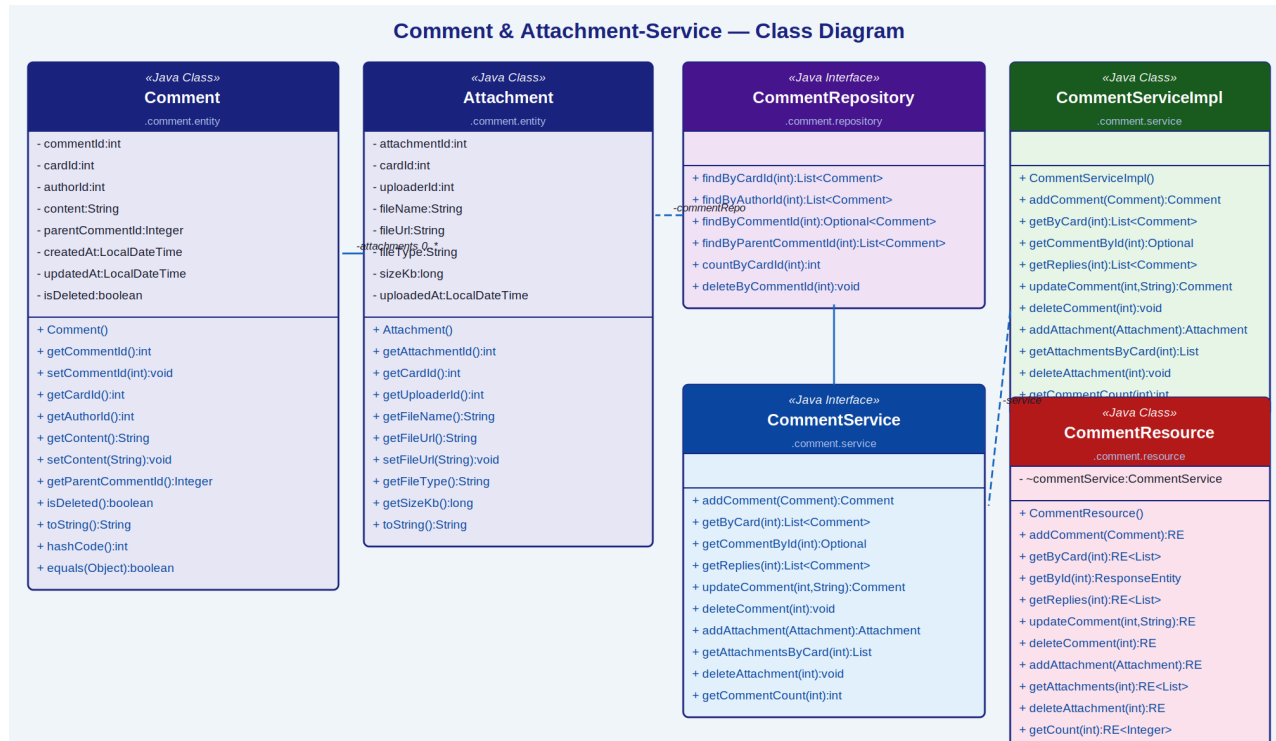


Figure 7: Comment & Attachment-Service — Class Diagram

Component	Type	Key Attributes / Methods
Comment	Java Class (Entity)	commentId, cardId, authorId, content, parentCommentId (nullable for top-level), createdAt, updatedAt, isDeleted
Attachment	Java Class (Entity)	attachmentId, cardId, uploaderId, fileName, fileUrl, fileType, sizeKb, uploadedAt — linked to Card (0..*)
CommentRepository	Java Interface	findByCardId(), findByAuthorId(), findByCommentId(), findByParentCommentId(), countByCardId(), deleteByCommentId()
CommentServiceImpl	Java Class	addComment(), getByCard(), getCommentById(), getReplies(), updateComment(), deleteComment(), addAttachment(), getAttachmentsByCard(), deleteAttachment(), getCommentCount()

Component	Type	Key Attributes / Methods
CommentService	Java Interface	Declares all comment and reply CRUD, attachment upload/deletion, and count operations
CommentResource	Java Class (REST)	Exposes /comments and /attachments endpoints: POST (add), GET (by card/id/replies), PUT (update), DELETE, GET count

4.7 Label & Checklist-Service

The Label & Checklist-Service manages two lightweight but important card enrichment features. Labels are board-scoped and colour-coded; a card can carry multiple labels for quick visual categorisation. Checklists are card-scoped and contain ordered ChecklistItems, each with a completion flag, optional assignee, and optional due date. The service computes checklist completion percentage for the card progress indicator.

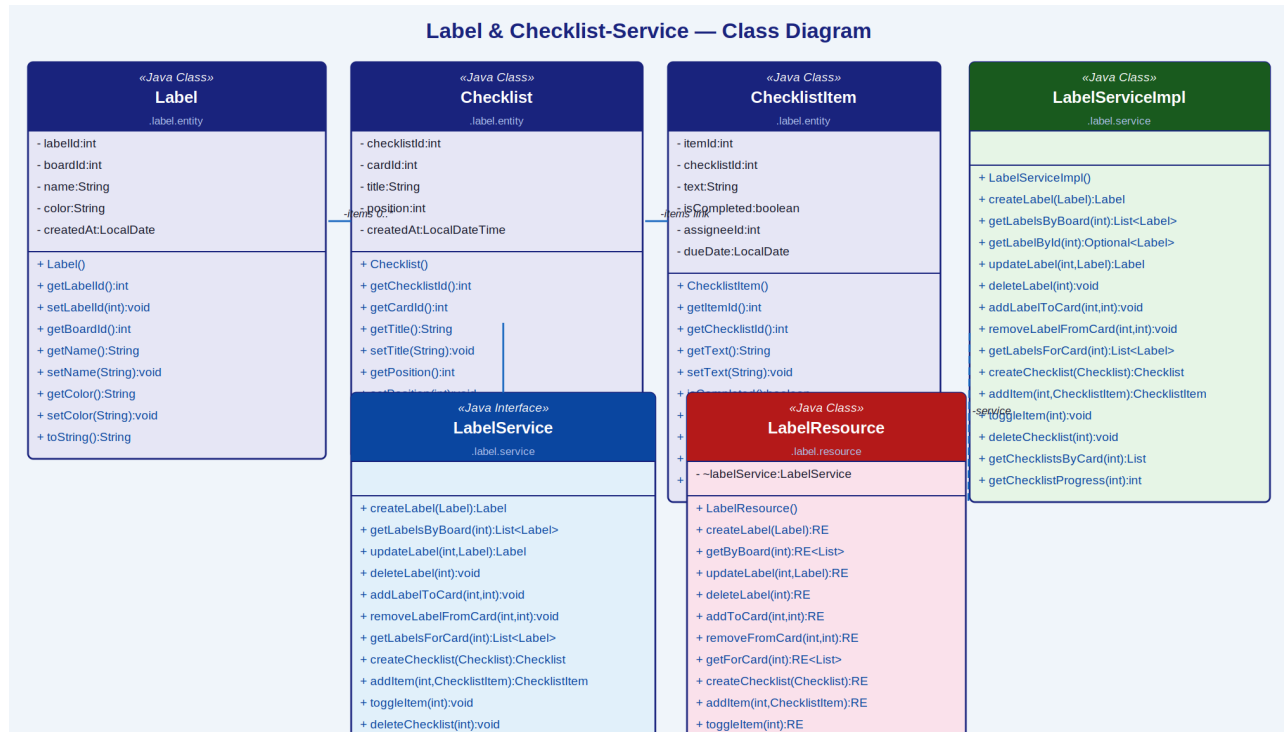


Figure 8: Label & Checklist-Service — Class Diagram

Component	Type	Key Attributes / Methods
Label	Java Class (Entity)	labelId, boardId, name, color (hex), createdAt — board-scoped, reusable across cards
Checklist	Java Class (Entity)	checklistId, cardId, title, position, createdAt — contains 0..* ChecklistItems
ChecklistItem	Java Class (Entity)	itemId, checklistId, text, isCompleted, assigneeId (optional), dueDate (optional)
LabelServiceImpl	Java Class	createLabel(), getLabelsByBoard(), updateLabel(), deleteLabel(), addLabelToCard(), removeLabelFromCard(), getLabelsForCard(), createChecklist(), addItem(), toggleItem(), deleteChecklist(), getChecklistsByCard(), getChecklistProgress()

Component	Type	Key Attributes / Methods
LabelService	Java Interface	Declares all label CRUD, card-label association, checklist CRUD, item toggle, and progress computation operations
LabelResource	Java Class (REST)	Exposes /labels and /checklists endpoints: POST, GET (by board/card), PUT, DELETE, POST/DELETE (card-label), POST (item), PUT (toggle), GET (progress)

4.8 Notification-Service

The Notification-Service dispatches and stores in-app and email alerts for all collaboration events: card assignment, @mention in comment, due date approaching (1 day and 1 hour), card moved to Done, and comment replies. Notifications carry an actorId, recipientId, relatedId (cardId/boardId), and relatedType for deep-linking. The service tracks read/unread state, exposes an unread badge count, and supports bulk broadcast for admin announcements and system-generated reminders.

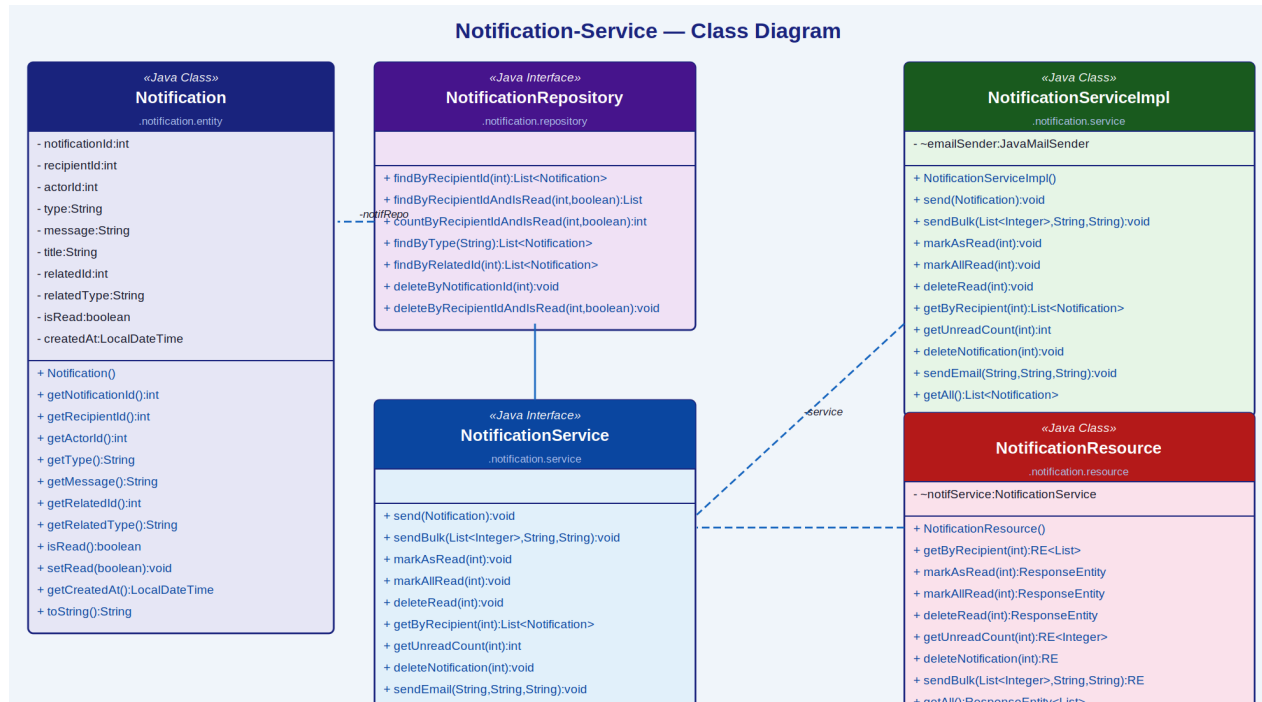


Figure 9: Notification-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, recipientId, actorId, type (ASSIGNMENT/MENTION/DUE_DATE/COMMENT/MOVE), message, title, relatedId, relatedType, isRead, createdAt
NotificationRepository	Java Interface	findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findByRelatedId(), deleteByNotificationId(), deleteByRecipientIdAndIsRead()
NotificationServiceImpl	Java Class	send(), sendBulk(), markAsRead(), markAllRead(), deleteRead(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), getAll()
NotificationService	Java Interface	Declares notification dispatch (single/bulk/email), retrieval, read-state management, and deletion operations

Component	Type	Key Attributes / Methods
NotificationResource	Java Class (REST)	Exposes /notifications endpoints: getByRecipient, markAsRead, markAllRead, deleteRead, getUnreadCount, delete, sendBulk, getAll

4.9 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC integration layer that wires all underlying microservices together and renders Thymeleaf views for members, board owners, and administrators. It contains three controllers — BoardViewController, CardController, and AdminController — each injecting relevant service dependencies via Spring `@Autowired` and calling REST Resource endpoints via `RestTemplate` for inter-service coordination.

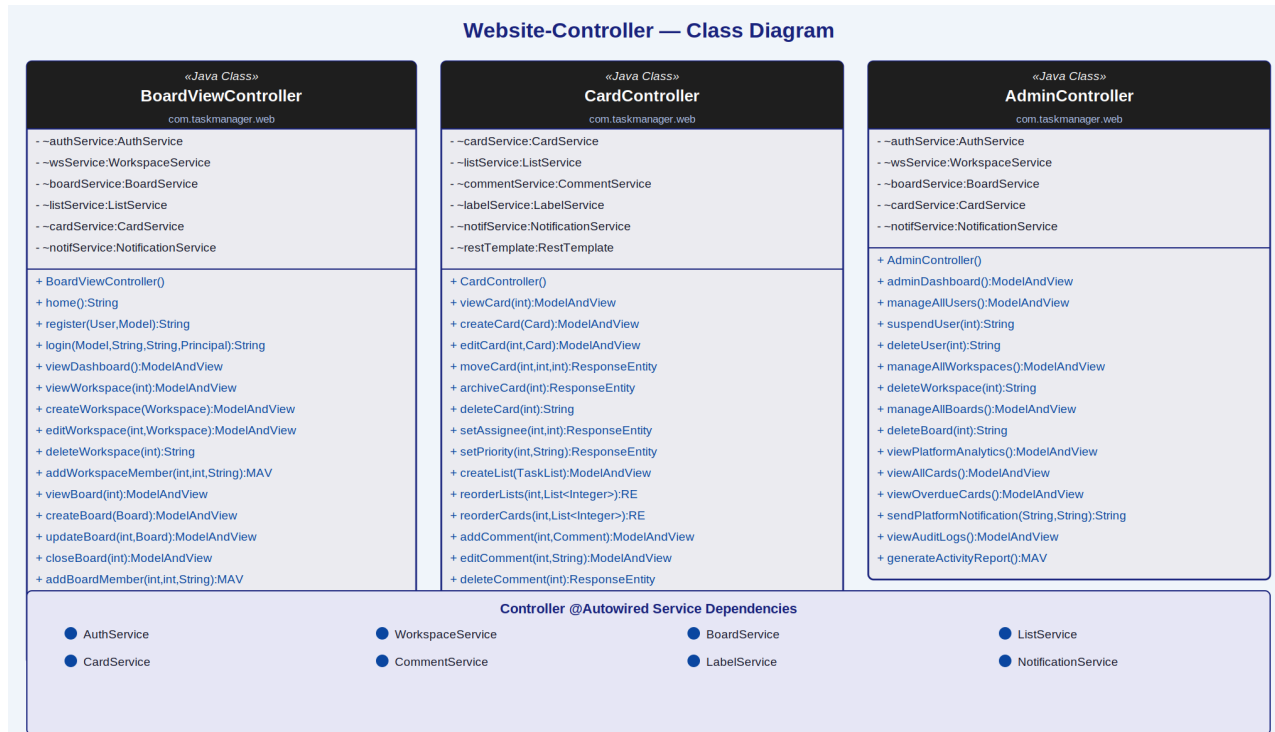


Figure 10: Website-Controller — Class Diagram

Controller	Type	Key Methods
BoardViewController	Java Class (MVC)	home(), register(), login(), viewDashboard(), viewWorkspace(), createWorkspace(), editWorkspace(), deleteWorkspace(), addWorkspaceMember(), viewBoard(), createBoard(), updateBoard(), closeBoard(), addBoardMember(), viewNotifications(), markNotifRead(), logout()
CardController	Java Class (MVC)	viewCard(), createCard(), editCard(), moveCard(), archiveCard(), deleteCard(), setAssignee(), setPriority(), createList(), reorderLists(), reorderCards(), addComment(), editComment(), deleteComment(), addAttachment(), deleteAttachment(), addLabel(), createChecklist(), toggleChecklistItem()

Controller	Type	Key Methods
AdminController	Java Class (MVC)	adminDashboard(), manageAllUsers(), suspendUser(), deleteUser(), manageAllWorkspaces(), deleteWorkspace(), manageAllBoards(), deleteBoard(), viewPlatformAnalytics(), viewAllCards(), viewOverdueCards(), sendPlatformNotification(), viewAuditLogs(), generateActivityReport()

5. Microservices Architecture Overview

FlowBoard is structured as a set of independently deployable microservices following the same layered pattern as the EShoppingZone reference. Each service owns its database schema, REST API contract, service interface, and business logic. Synchronous inter-service calls use RestTemplate; asynchronous notification events are routed via RabbitMQ.

Microservice	Base Package	Primary Domain
auth-service	com.flowboard.auth	User registration, login, JWT, OAuth2 (Google/GitHub)
workspace-service	com.flowboard.workspace	Workspace CRUD, workspace membership and roles
board-service	com.flowboard.board	Board CRUD, board state (open/closed), board membership
list-service	com.flowboard.list	List/column CRUD, position management, archival, board transfer
card-service	com.flowboard.card	Card CRUD, move/reorder, priority, status, overdue detection
comment-service	com.flowboard.comment	Threaded comments, replies, file attachments
label-service	com.flowboard.label	Board labels, card-label associations, checklists and items
notification-service	com.flowboard.notification	In-app and email notifications, bulk dispatch, read-state
flowboard-web	com.flowboard.web	Spring MVC controllers, Thymeleaf view rendering

6. Non-Functional Requirements

Category	Requirement
Performance	Board load (lists + cards) must respond within 1 second for boards with up to 500 cards; drag-and-drop position commits within 200ms
Scalability	Each microservice independently scalable via Docker/Kubernetes; card position cache in Redis for high-frequency drag-and-drop operations
Security	Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for social login; HTTPS enforced; board access gated by membership check
Availability	99.9% uptime SLA; health-check endpoints per service; graceful degradation if a non-critical service is unavailable
Data Integrity	Card position reordering is atomic to prevent position conflicts; concurrent move operations use optimistic locking
Real-time Updates	WebSocket (STOMP) broadcasts card moves, comment additions, and checklist toggles to all active board viewers in real time
Drag-and-Drop UX	Client-side drag-and-drop uses optimistic UI updates; server position commit happens in the background with rollback on failure
Audit Trail	All significant actions (card create/move/delete, member role change) are logged with actor, timestamp, and before/after values
Usability	Responsive Kanban UI on desktop and tablet; WCAG 2.1 AA accessibility compliance; keyboard-navigable card drag-and-drop
Maintainability	Services independently deployable; RESTful API contracts versioned under /api/v1/

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA)
Authentication	JWT (JSON Web Tokens), Spring Security OAuth2, Google / GitHub OAuth
Database	MySQL (primary relational store per service); Redis (card position cache, session store, real-time board state)
Real-time Sync	WebSocket (STOMP) via Spring WebSocket for live board updates, comment additions, and drag-and-drop sync
Search	Elasticsearch for cross-board card and label search by keyword, assignee, or due date range
File Storage	AWS S3 for card attachment uploads; CloudFront CDN for file delivery
Frontend	React.js (SPA) with react-beautiful-dnd for drag-and-drop; Redux for board state management; Tailwind CSS for UI
Messaging	RabbitMQ for asynchronous notification dispatch and due-date reminder scheduling
Scheduled Jobs	Spring @Scheduled / Quartz for overdue card detection, due-date reminders, and expired-session cleanup
Containerisation	Docker with Docker Compose; Kubernetes for production autoscaling and rolling deployment
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints across all services
CI/CD	GitHub Actions or Jenkins pipeline for automated build, test, and deployment

8. Glossary

Term	Definition
Workspace	The top-level organisational container that groups related boards and team members
Board	A Kanban-style project board containing an ordered set of lists and cards
List	A vertical column on a board representing a workflow stage, e.g. To Do, In Progress, Done
Card	The primary task unit on a board — carries title, description, priority, assignee, due date, and rich metadata
Position	An integer ordering field on lists and cards used to render them in the correct left-to-right or top-to-bottom order
Priority	A card attribute indicating urgency: Low, Medium, High, or Critical
Status	A card's workflow stage: To Do, In Progress, In Review, or Done
Overdue	A card is overdue when the current date exceeds its due date and its status is not Done
Label	A named, colour-coded tag scoped to a board, attachable to any card for visual categorisation
Checklist	A sub-task list attached to a card, containing ChecklistItems each with a completion toggle
Drag-and-Drop	The user interaction pattern for reordering cards within a list or moving them between lists
Archiving	A soft-delete operation that hides a card or list from normal view while retaining it for restoration
BoardMember	A user associated with a specific board with a defined role (Observer, Member, or Admin)
WorkspaceMember	A user associated with a workspace with a role of Admin or Member
JWT	JSON Web Token — a compact, stateless token used for authentication across all services
STOMP	Simple Text Oriented Messaging Protocol — used over WebSocket to broadcast real-time board updates